

Web Testing Project

Quality Assurance — Graduation Project

*Prepared by: Abanob Sorour
Mahmoud Bauyomi*

*Under the Supervision of
Dr. Amany Shousha
Eng. Aya Fawaz*

48

Test Cases

41

Passed

7

Defects

85.4%

Pass Rate

AGENDA

01

Project Overview

What is Automation Exercise? Goals & Scope

03

Tool Comparison

Why we chose these tools over the alternatives

05

Test Coverage & Execution

48 test cases across 5 functional modules — full breakdown

07

Test Plan Highlights

Entry/exit criteria, risks, recommendations

02

Technology Stack

Manual (Excel · Chrome) vs Automation (Java · Selenium · TestNG · Maven · Allure)

04

Test Plan & Strategy

Test Plan highlights · BVA · EP · Negative Testing · 5 Code Modules

06

Defects Found

7 defects with root causes

08

Conclusion

Results summary & next steps

01 | PROJECT OVERVIEW

What is Automation Exercise?

automationexercise.com

is a fully functional e-commerce web application designed specifically for QA automation practice. It provides a realistic environment to test:

- User Registration & Account Management
- Product Browsing & Shopping Cart
- Search & Filter Functionality
- Subscription
- Login, Logout & Delete Account

Project Goal

Design and execute a comprehensive test suite covering all key user journeys — validating correct behavior, catching input-validation defects, and ensuring production-readiness.

In Scope

- Registration / Signup (2 steps)
- Login · Logout · Delete Account
- Cart: Add · Remove · Quantity
- Subscription Email Validation
- Product Search

02 | TECHNOLOGY STACK



Manual Testing

Excel Sheet

Test Management

Human-readable test case repo — TC ID, steps, expected vs actual, pass/fail. Easy to share and review.

Chrome Browser

Test Environment

All manual tests executed on Chrome for Windows — the most widely used browser, ensuring real-world coverage.



Automation Testing

Java

Test Language

Strongly typed, OOP support for Page Object Model, industry standard for Selenium.

Selenium

Browser Automation

Simulates real user interactions — clicks, typing, form submission across browsers.

TestNG

Test Framework

Parallel execution, grouping, data providers, and rich annotations (@BeforeSuite etc.).

Maven

Build & Dependency

One POM file manages all dependencies — reproducible builds, no JAR conflicts.

Allure

Reporting

Interactive HTML report: step-by-step logs, screenshots on failure, trends dashboard.

03 | WHY THESE TOOLS — NOT THE ALTERNATIVES

Java vs Python

✓ We used: Java

- Industry standard for Selenium
- Strong OOP support for Page Object Model
- Excellent IDE support (IntelliJ)
- Strong typing catches errors early

✗ Not: Python

- Dynamic typing leads to runtime errors
- Less common in enterprise QA teams
- Weaker TestNG/Maven integration

TestNG vs JUnit

✓ We used: TestNG

- Built-in data providers for DDT
- Test grouping & dependency support
- Parallel test execution out of box
- Richer annotations (@BeforeSuite etc)

✗ Not: JUnit

- No built-in parallel execution
- Weaker test grouping support
- Requires Parameterized extension for DDT

Selenium vs Playwright

✓ We used: Selenium

- Industry standard for web automation
- Huge ecosystem & community support
- Excellent Page Object Model support
- Mature integration with Java, TestNG & Maven

✗ Not: Playwright

- Newer ecosystem compared to Selenium
- Less common in existing enterprise Java QA stacks
- Smaller pool of Selenium-style learning resources

04 | TEST PLAN & STRATEGY

Environment: Chrome · Windows · automationexercise.com

Entry Criteria: App live · Selenium configured · TCs approved

Exit Criteria: All 48 TCs executed · Defects logged · Pass rate $\geq$ 80%

Code Modules: Register · confirmationUser · Products · deleteAccount · validationErrorMessage

EP

Equivalence Partitioning

Divided inputs into valid/invalid classes. One value per partition tested — e.g. valid email, empty email, email missing @.

BVA

Boundary Value Analysis

Tested edges of valid ranges — empty fields, spaces-only, max-length inputs — where most bugs hide.

NEG

Negative Testing

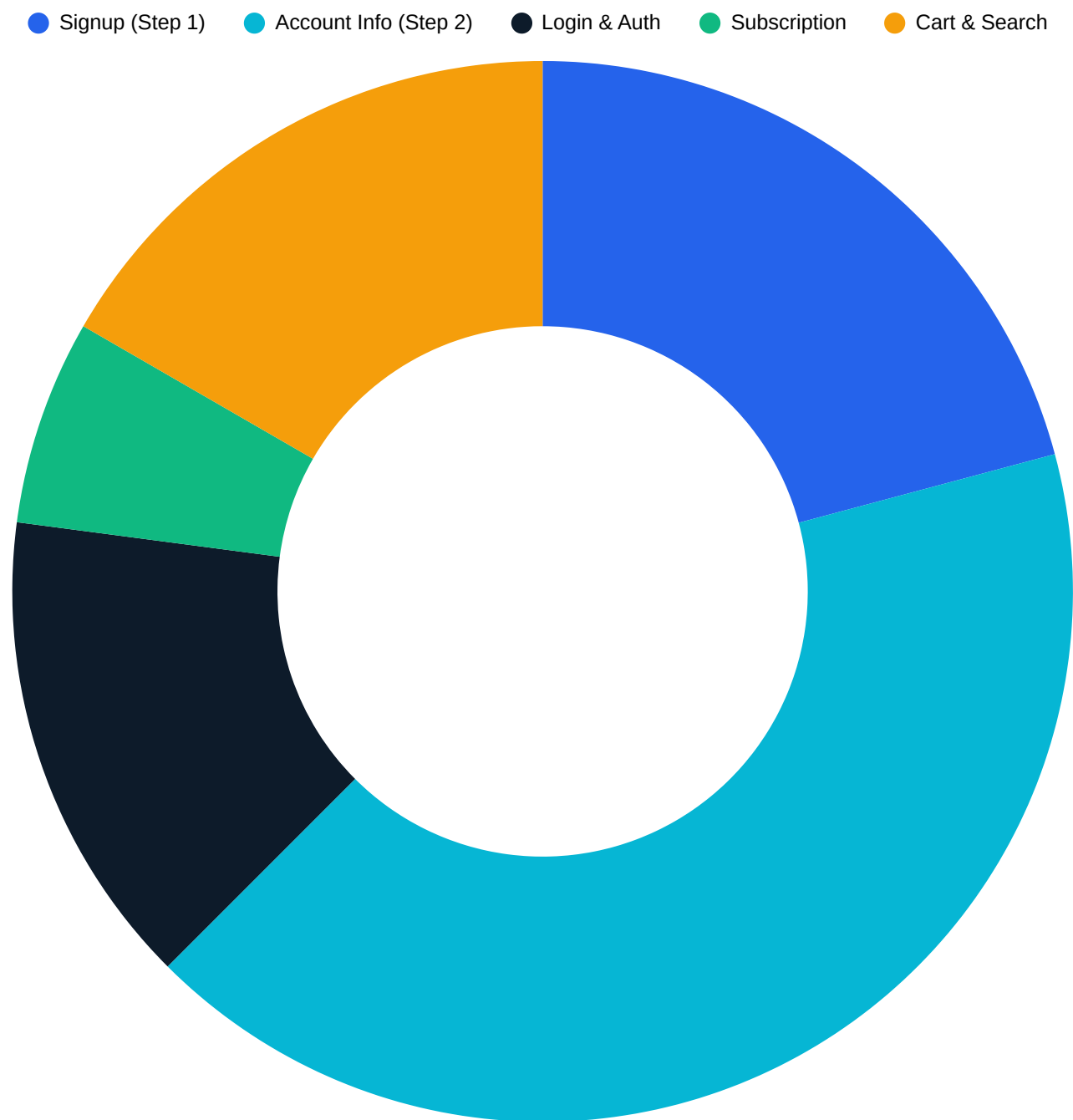
Systematically fed invalid data to every field: special chars, Arabic names, alphabetic phone numbers, duplicate emails.

E2E

End-to-End Testing

Full user journeys: Signup Step 1 → Account Info → Login → Browse → Cart. No isolated unit tests.

05 | TEST Case Design



A total of 48 test cases covering positive & negative scenarios across 5 API modules.

Module	TCs	Pass	Fail
Signup — Step 1	10	9	1
Account Info — Step 2	20	14	6
Login & Auth	7	7	0
Subscription	3	3	0
Cart & Search	8	8	0
TOTAL	48	41	7

06 | TEST PLAN HIGHLIGHTS

Entry & Exit Criteria

Entry (before testing begins)

- App accessible at automationexercise.com
- Chrome + Selenium configured
- All TCs reviewed & approved
- Test data prepared

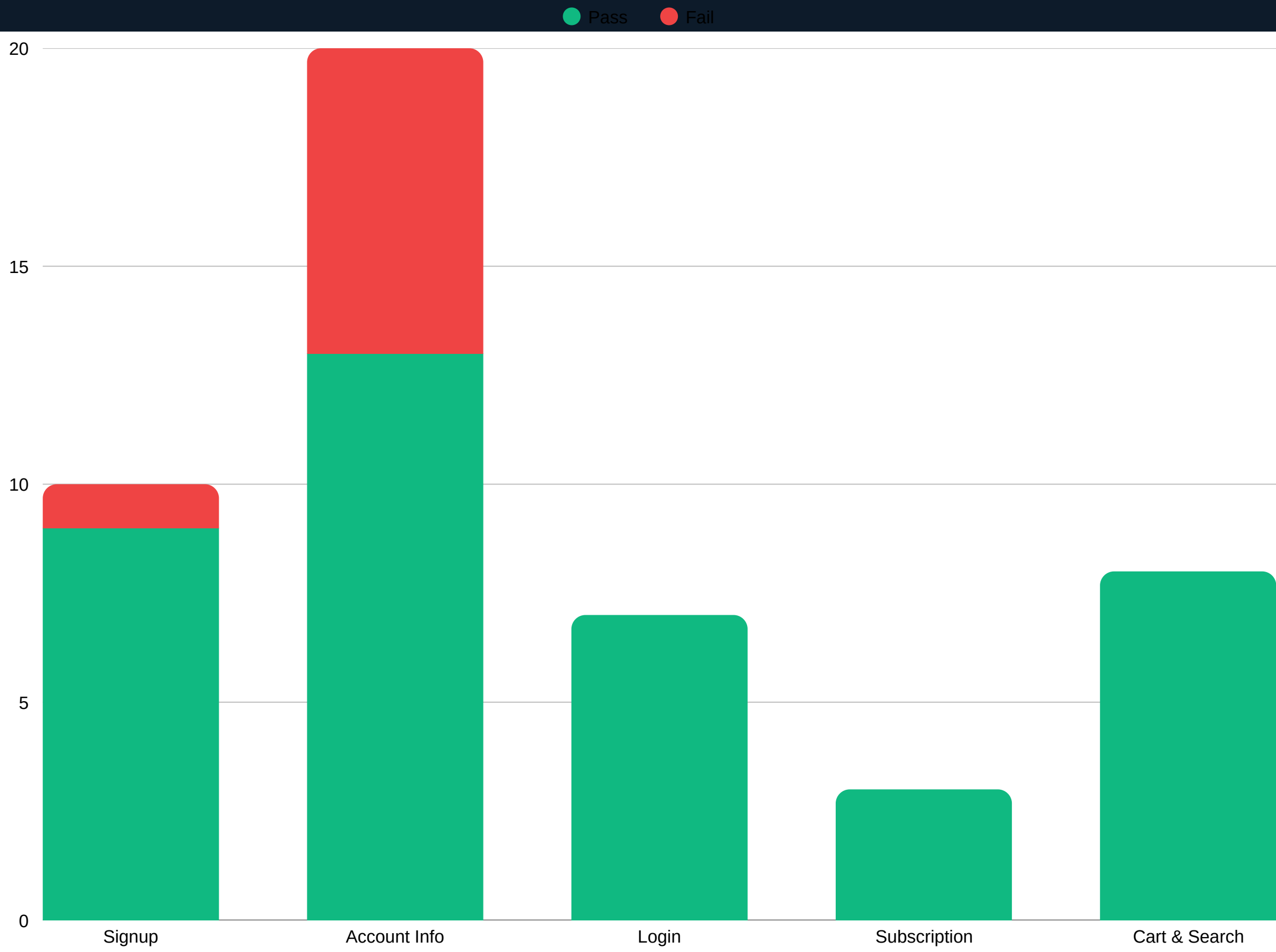
Exit (testing complete when...)

- All 48 TCs executed
- All defects logged with severity
- Pass rate $\geq 80\%$ achieved ✓ (85.4%)
- Test Summary Report ready

Recommendations

- 01** Enforce phone validation — digits only + optional leading + **Critical**
- 02** Sanitize name fields — reject spaces-only & special-char inputs **Critical**
- 03** Add server-side validation — all current validation is client-side only **High**
- 04** Expand to Firefox, Edge, Safari — Chrome-only is insufficient for release **Medium**

Test Results



48
Total TCs

48
Executed

41
Passed

7
Failed

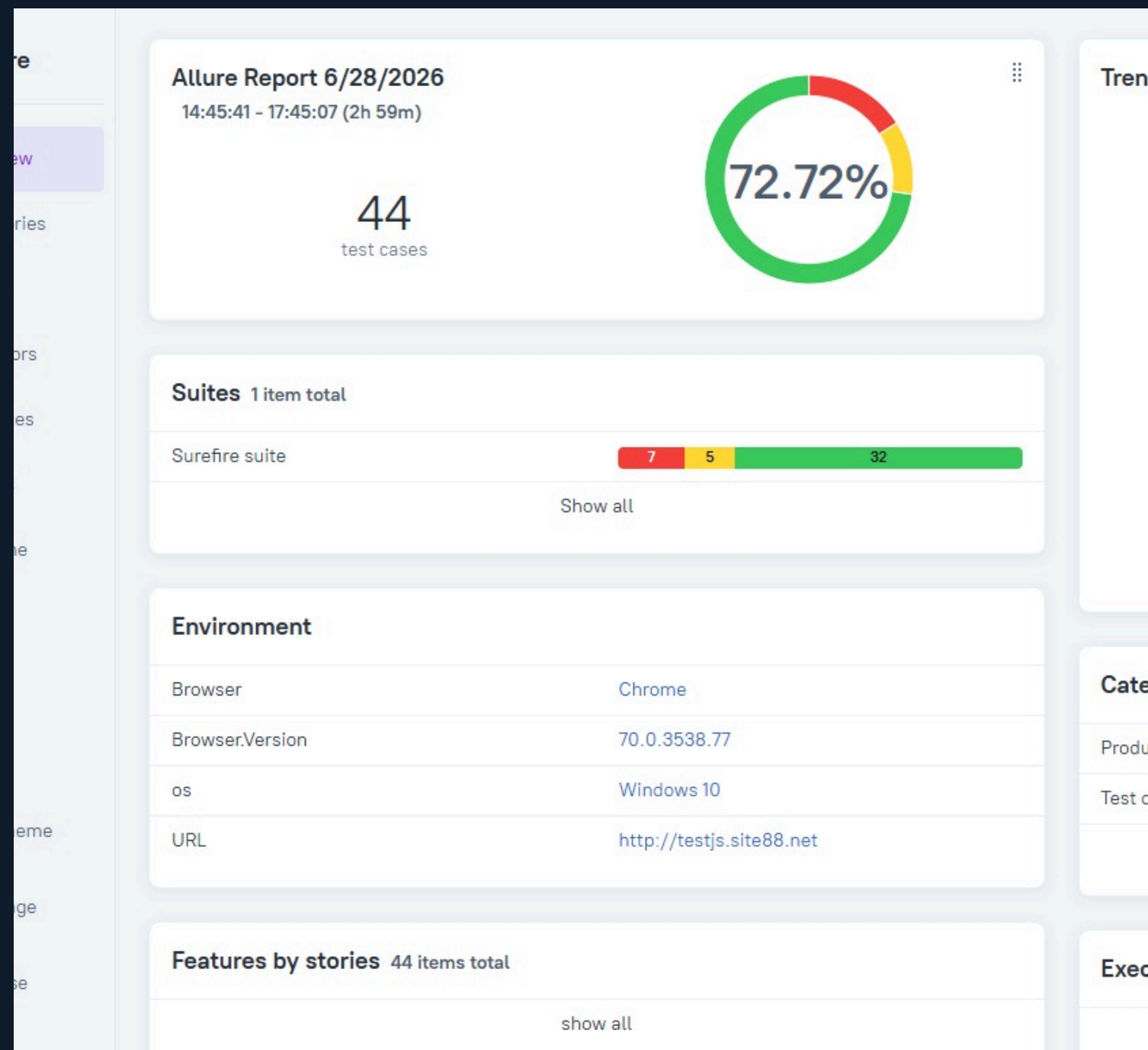
85.4%
Pass Rate

3
Critical Bugs

4
High Bugs

100%
Login/Cart/Sub

+ Allure Report Overview



```
allure-results
├── 0c619e02-87c0-48c7-af11-6a6a415b2cb5-container.json
├── 2a6b1592-753b-4e23-9685-e8ab09d24200-result.json
├── 2b1e7e11-b053-46b9-bf56-002505b77a2e-result.json
├── 2b4cbd6a-1041-4d83-b9dc-3a28d97ce817-result.json
├── 2fb60bb6-6b54-4333-9d51-c4ad858b7e8f-result.json
├── 3cf2ff02-3bc5-41cf-ba34-549a43452ef1-result.json
├── 04bccb59-23c8-45ca-9263-e5d331c4e841-result.json
├── 5b1d3915-6a24-4ef2-90e0-5e300faac7a3-result.json
├── 5da9d9ef-34c3-4907-bda9-f0e82eb33178-result.json
├── 5fe8a00a-532c-4da1-a846-cfd120ee9071-result.json
├── 12e098fa-85b6-4e72-8a94-8c2820a1cbe7-result.json
└── 37ebfd59-cab3-4f18-803e-596e25f69f8d-result.json
```

A screenshot of the Godot 4.0.3 interface. The top bar is dark grey with the text 'Godot 4.0.3' on the left and a window icon on the right. Below the top bar, the interface is divided into several panels. On the left is the 'Audio Mixer' panel, which has a dark background and two vertical sliders labeled 'Global' and 'Mic/Aux'. The 'Global' slider is set to 0.0 dB and the 'Mic/Aux' slider is also set to 0.0 dB. Below the sliders are two sets of speaker icons. In the center is a large window titled 'Studio Mode' with a dark background. To the right of the 'Studio Mode' window is a 'Settings' panel with a dark background and a list of settings. Below the 'Settings' panel is a 'Scenes' panel with a dark background and a list of scenes. The 'Scenes' panel has a white header and a blue button labeled 'Scene'. In the background, two web browser windows are visible: 'Allure حل مشكلة تقرير' and 'Google Chrome'.

Windows taskbar showing search bar, taskbar icons (File Explorer, Edge, Word, etc.), system tray (clock, date, network, volume), and language (ENG).

CONCLUSION

Solid foundation. One fixable gap.

- 85.4% pass rate — all core flows (Login, Cart, Search, Subscription) are production-ready.
- 7 defects share one root cause: missing input sanitization on name & phone fields — one fix pattern closes all bugs.
- Selenium + Java + TestNG + Maven stack delivered reliable, repeatable, reportable automation.
- Allure Report gave us a professional interactive dashboard — every failure documented with full steps and actual result.

+ GitHub Repo



Test Case Report

Thank you so much

 +201270180191

 abanob.soror2017@gmail.com